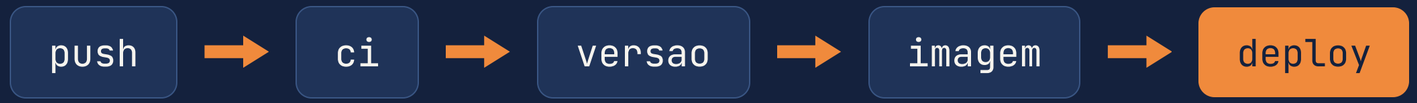
DEVOPS · TRABALHO PRÁTICO 2

CI/CD e Observabilidade

Leilão de Bois em microsserviços: do commit ao ambiente de homologação



Lucas · auth e usuarios **Thiago** · leiloes **Anderson** · lances

24/10/2026

A ESPECIFICAÇÃO

O que o T2 pedia

1 — **Git Flow**
feature/* → develop → main

2 — **CI**
build, testes e SonarCloud

3 — **CD no Render**
develop → DEV
main → HOMOL

4 — **Ambientes**
Swagger só no DEV

6 — **Containers**
Dockerfile, build e GHCR

7 — **Versionamento**
Semantic Versioning

8 — **Segurança**
Dependabot e secrets do GitHub

9 — **Observabilidade**
Prometheus e Grafana

10 — **Arquitetura**
1 pipeline por serviço

11 — **Entregáveis**
URLs, dashboards e README

> Os 10 itens estão **cumpridos** e foram conferidos com evidência em 20/10. A especificação pula do item 4 para o 6.

A EQUIPE

Quem fez o quê

Thiago

`leiloes-service`

- Criou a develop e o primeiro pipeline
- O modelo dos outros três: workflow, deploy, versão, métricas e dashboard
- Neon e Render do leiloes

Lucas

`auth-service e usuarios-service`

- Pipelines, métricas e dashboards dos dois (PR #13)
- Admin do repositório: environments, secrets e variáveis
- Neon e Render dos dois serviços

Anderson

`lances-service`

- Pipeline, métricas da Saga e dashboard (PR #16)
- Segurança sem o Kong, Node 24 e integração com o T1
- As duas entregas para a main e os testes na nuvem

> Tudo o que é infraestrutura **espelha o leiloes**: os 4 pipelines funcionam e se explicam do mesmo jeito.

LINHA DO TEMPO

Dez dias, do primeiro pipeline à segunda entrega

14/10

Thiago cria a develop e o pipeline do leiloes, o modelo dos outros

15/10

Pipelines do auth e do usuarios (#13) e revisão do T1 integrada (#14)

16/10

Pipeline do lances (#16), jwt.verify (#18), Node 24 (#20), chave interna e LGPD (#21)

17/10

Regras do Git Flow registradas no CLAUDE.md (#23)

18–19/10

Neon e Render: 8 serviços, 8 environments e os segredos de cada ambiente

20 e 24/10

Entregas develop → main (#30 e #49): HOMOL no ar e teste integrado 21/21

16

PRs da equipe juntados

30

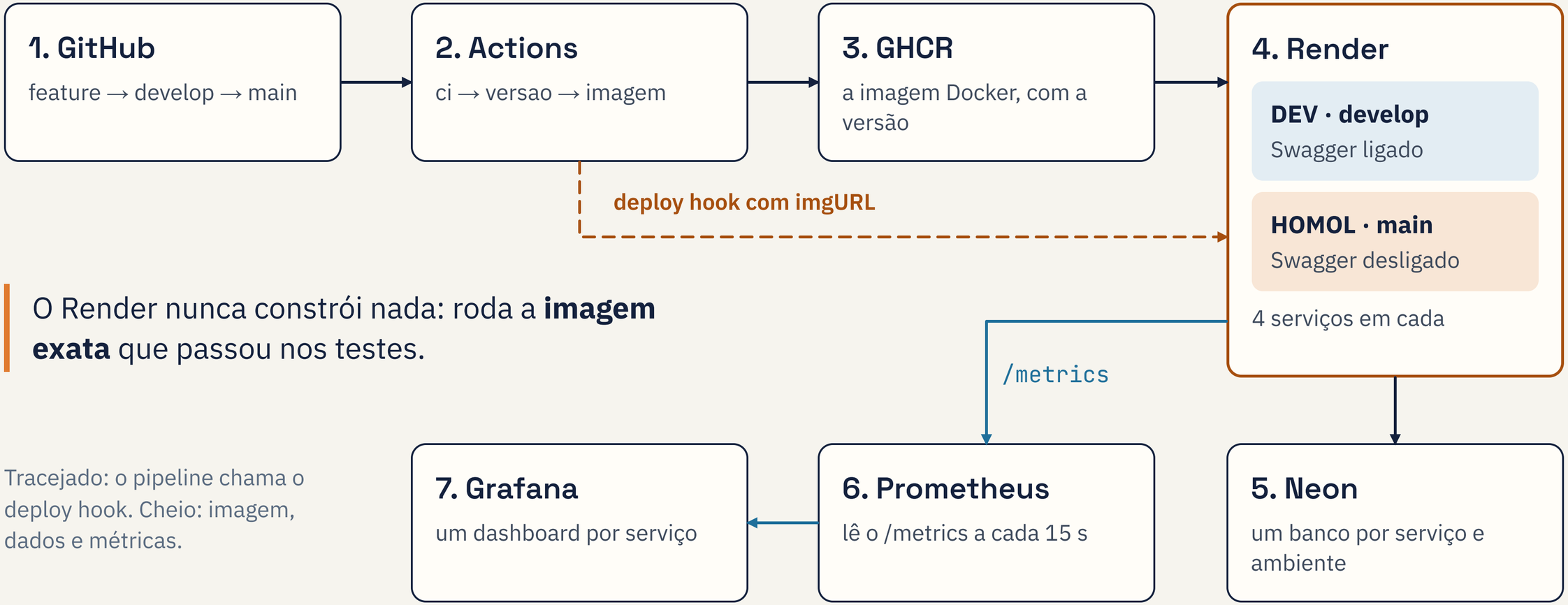
PRs do Dependabot decididos

2

entregas da develop para a main

VISÃO GERAL

Do commit à nuvem

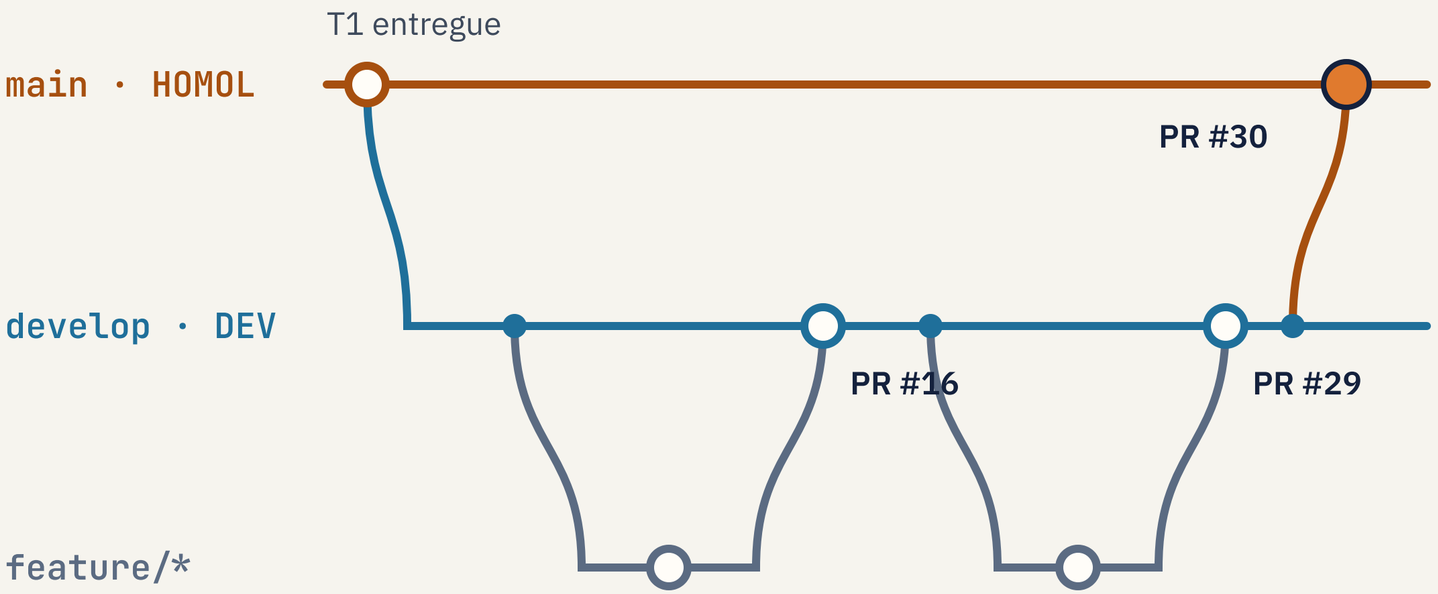


> O Render nunca constrói nada: roda a **imagem exata** que passou nos testes.

Tracejado: o pipeline chama o deploy hook. Cheio: imagem, dados e métricas.

ITEM 1 · GIT FLOW

Toda mudança passa por uma feature e um PR



Regras da equipe

- Toda tarefa numa feature/<nome>, criada da develop
- Entra por Pull Request, com merge commit
- develop vai para o DEV; main, para o HOMOL
- As feature/* não são apagadas: são a evidência
- O Dependabot também segue o fluxo

Exemplo real: o pipeline do lances (PR #16), as URLs reais (PR #29) e a entrega para a main (PR #30).

ITENS 2 E 10 · CI

O pipeline, job a job

ci

- npm ci: o lock exato
- npm audit: crítica falha
- Jest, cobertura ≥ 50%
- SonarCloud

versao

- lê o package.json
- decide tags e ambiente
- na main: trava de versão

imagem

- docker build em todo push
- publica no GHCR (develop e main)
- cria a tag Git na main

deploy-dev

develop → Render DEV, com smoke test no /health

deploy-homol

main → Render HOMOL, com smoke test no /health

paths: um workflow por serviço. Um erro no lances não trava o deploy do auth.

needs: nada é publicado nem implantado se um teste falhar. Em PR roda tudo, sem publicar.

ITEM 7 · VERSIONAMENTO

Semantic Versioning, com trava

1 . **1** . **1**

MAJOR
quebra quem usa a API

MINOR
funcionalidade nova

PATCH
correção ou dependência

> A versão mora no **package.json** de cada serviço. O leiloes foi para 1.2.0 com as regras do T1; na segunda entrega, os quatro subiram o PATCH.

Branch	Tags da imagem	Exemplo
develop	X.Y.Z-dev.N, dev, sha	1.1.1-dev.56
main	X.Y.Z, homol, latest, sha e a tag Git	1.1.1

Trava de versão

Na main, se a tag lances-service-v1.1.0 já existe em outro commit, o job versao falha. Uma versão nunca aponta para dois códigos.

```
npm version patch --no-git-tag-version
```

ITEM 6 · CONTAINERIZAÇÃO

Uma imagem por serviço, publicada no GHCR

O Dockerfile

`node:24-alpine`
Node 24 LTS, com suporte até 30/04/2028

`npm ci --omit=dev`
build reprodutível, sem dependências de teste

`USER node`
o processo não roda como root

`HEALTHCHECK`
o container confere o próprio /health

`ARG VERSAO_IMAGEM`
a versão completa aparece no /health e nas métricas

`ghcr.io/ratzmann/lances-service`

main

`1.1.1` `homol` `latest` `sha-bdf6271`

develop

`1.1.1-dev.56` `dev` `sha-b005c46`

> O login no GHCR usa o token automático do Actions: nenhuma senha de registro guardada.

ITEM 3 · CD NO RENDER

Deploy que se confere sozinho

1

Chama o deploy hook

com imgURL = a imagem da versão

→

2

O Render baixa a imagem

do GHCR e reinicia o serviço

→

3

Smoke test

GET /health a cada 10 s, por até 10 min

→

4

A versão bateu?

sim: confirmado. Não: o job falha

> Por que conferir a **versão**, e não só o 200? Logo depois do hook, a versão antiga ainda está no ar e também responde 200.

```
# log real: deploy HOMOL do leiloes, 20/10
Disparando deploy de ghcr.io/ratzmann/leiloes-service:1.2.0
tentativa 1: versao no ar = 1.2.0 (esperada 1.2.0)
Deploy confirmado: https://leiloes-service-homol-igr9.onrender.com esta com a versao 1.2.0
```

ITEM 4 · AMBIENTES

DEV x HOMOL: mesma imagem, outra configuração

	DEV	HOMOL
Recebe	merge na develop	entrega na main
Versão	1.1.0-dev.N	1.1.0
Swagger	ligado em /docs	desligado (404)
Banco	lances_dev	lances_homol
Segredos	par do DEV	par do HOMOL
Falha simulada	permitida	desligada

> O Swagger depende de **APP_ENV** e é fail-closed: sem a variável, fica desligado. No HOMOL a rota /docs nem é registrada.

`lances-service-dev /docs`

Swagger UI · lances-service

`POST /lances` `GET /lances/:id`

`GET /lances/sagas`

`lances-service-homol /docs`

404 a rota não existe no HOMOL

ITEM 11 · ENTREGÁVEIS

8 serviços no ar, no Render

Serviço	DEV (develop)	HOMOL (main)
auth	`auth-service-dev-rslq.onrender.com`	`auth-service-homol-czky.onrender.com`
usuarios	`usuarios-service-dev.onrender.com`	`usuarios-service-homol.onrender.com`
leiloes	`leiloes-service-dev-bnhz.onrender.com`	`leiloes-service-homol-igr9.onrender.com`
lances	`lances-service-dev.onrender.com`	`lances-service-homol.onrender.com`

```
GET /health → {"status":"ok","service":"lances-service","versao":"1.1.1","ambiente":"homol"}
```

Neon Postgres

um banco por serviço e por ambiente; a DATABASE_URL fica só no Render

Plano gratuito

o serviço dorme após ~15 min; a primeira chamada leva até 1 min

ITEM 8 · SEGURANÇA

Na nuvem não há Kong na frente

No docker-compose (T1)

Cliente → **Kong confere o token** → serviços

No Render (T2)

Cliente → **cada serviço, com URL pública**

`jwt.verify`

usuarios, leiloes e lances conferem a assinatura do token com o JWT_SECRET do auth

token inválido ou vencido: 401

`X-Chave-Interna`

as rotas internas do usuarios (perfil e crédito) só atendem quem manda a chave

sem a chave: 403

`LGPD do crédito`

o resumo de crédito só aparece para o próprio licitante

outro usuário: 403

Secrets: SONAR_TOKEN no repositório; os deploy hooks nos environments; DATABASE_URL, JWT_SECRET e CHAVE_INTERNA só no Render. Nada disso está no código.

ITEM 8 · DEPENDABOT

O robô sugere, a equipe decide

30

PRs abertos pelo robô

21

juntados, com CI verde

9

fechados com justificativa

Atualização	Decisão	Por quê
GitHub Actions	juntadas	checkout, setup-node, Docker e Sonar: sem impacto no código
pg, dotenv 18, Jest, supertest, bcryptjs 3	juntadas	CI verde e teste integrado na nuvem; no bcryptjs, login com senha antiga testado
Node 25 e 26	fechadas	só versões LTS; o Node 26 vira LTS em 28/10/2026
Express 5	fechadas	não migrar um serviço só; se migrar, os 4 juntos

> Toda segunda-feira o robô confere npm, Docker e Actions de cada serviço e abre os PRs para a develop. Ele lê a configuração da main: por isso os PRs de auth, usuarios e lances só vieram depois da entrega.

ITEM 9 · OBSERVABILIDADE

Cada serviço mede o próprio negócio

auth

`auth_registros_total`
`auth_logins_total{resultado}`

registros por papel; logins com sucesso x falha

usuarios

`usuarios_cadastros_total`
`usuarios_reservas_credito_total{operacao}`

cadastros; crédito reservado e liberado pela Saga

leiloes

`leiloes_cadastrados_total`
`leiloes_mudancas_status_total{status}`

leilões criados; o ciclo de vida de cada leilão

lances

`lances_registrados_total`
`lances_sagas_total{status}`

lances gravados; sagas concluídas, compensadas ou com pendência

`/metrics` → Prometheus, a cada 15 s → Grafana

Cardinalidade: a rota entra como /lances/:id, nunca /lances/42. Senão cada id vira uma série nova.

ITEM 9 · PROMETHEUS E GRAFANA

Um dashboard por serviço, pronto ao subir



Captura real do dashboard do lances em 24/10, com a versão da segunda entrega nos três ambientes. Os 5xx são a falha simulada que mostra a compensação.

Stack	Grafana
leiloes	:3000
auth	:3011
usuarios	:3012
lances	:3013

Datasource e dashboard provisionados: sobe com docker compose, sem clicar em nada. A captura de cada um está no README do serviço.

TESTES E VERIFICAÇÃO

Cinco níveis de teste

410
testes unitários, cobertura de 85% a 91%

75
passos ponta a ponta, pelo Kong

65
requisições no Postman, 98 conferências

8
serviços com smoke test a cada deploy

21/21
teste integrado na nuvem, no DEV e no HOMOL

O teste integrado passa pelos 4 serviços no Render

1. registro: auth → usuarios → 2. login → 3. leilão: leiloes → usuarios → 4. lance de A pela Saga →
5. B cobre; o crédito de A volta → 6. lance baixo: 400 · crédito alheio: 403 → 7. cancelar devolve o crédito

O QUE DEU ERRADO

Problemas e como resolvemos

Problema	Solução
Node 20 saindo de suporte	Node 24 LTS nos 4 serviços (PR #20); Node 25 e 26 recusados
Um serviço fazendo deploy no outro	environments com prefixo: auth-dev, lances-homol...
Serviços públicos, sem o Kong	jwt.verify, chave interna e LGPD do crédito (PRs #18 e #21)
O leiloes foi recriado e mudou de URL	URLs em variáveis; README e Prometheus atualizados (PR #29)
Deploy HOMOL do leiloes falhando	o secret tinha espaço e depois era o hook de um serviço apagado (404): hook certo e Re-run failed jobs
Ids de leilão reaproveitados no DEV	lances antigos apareceram num leilão novo; os ids seguintes não colidem
Dashboard local do leiloes com 500	JWT_SECRET na stack e séries que nascem com zero (PR #47)

> **Lição:** URL e segredo moram em variável, e todo deploy confere a própria versão. Foi isso que mostrou cada erro na hora.

RESULTADO

Os 10 itens cumpridos, com evidência

4

pipelines verdes, na develop e na main

8

serviços no ar: 4 no DEV e 4 no HOMOL

410

testes unitários no pipeline

21/21

teste integrado no DEV e no HOMOL

T2 · CI/CD E OBSERVABILIDADE

Obrigado!

Perguntas?

Para ler depois, no repositório

`docs/T2-PROCESSO.md`	este processo, por escrito
`<servico>/docs/T2-DECISOES.md`	as decisões de cada serviço
`README.md`	as URLs, como rodar e como testar

Lucas · auth e usuarios **Thiago** · leiloes **Anderson** · lances